

Assignment-15.4

Name: Sushma Purella

H.No:2303A52188

Batch:44

Task 1: Initial API Setup for a Service Platform

Prompt:

Generate a basic Flask application in Python.

Requirements:

- Create a Flask app.
- Add a route '/status'.
- The route should return a JSON response: {"status": "API is running"}.
- Run the app on localhost with debug mode enabled.
- Keep the code simple and beginner-friendly.

Code:

```
assgn15.4 task1.py > ...
1  from flask import Flask, jsonify
2
3  # Create Flask application
4  app = Flask(__name__)
5
6  # Create /status endpoint
7  @app.route('/status')
8  def status():
9      return jsonify({"status": "API is running"})
10
11 # Run the server
12 if __name__ == '__main__':
13     app.run(debug=True)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task1.py"
* Serving Flask app 'assgn15.4 task1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 107-307-218
```

127.0.0.1:5000/status

Pretty print ☐

```
{
  "status": "API is running"
}
```

Observation:

1. The Flask application was successfully created and executed.
2. The server started without any errors in VS Code.
3. The /status endpoint was tested using a browser.
4. The API returned the correct JSON response.
5. This confirms that the backend service is running properly

Task 2: Create API to Add and View Resources

Prompt:

Generate a Flask API in Python.

Requirements:

- Create an in-memory list to store service requests.
- Implement a GET endpoint '/requests' to retrieve all requests.
- Implement a POST endpoint '/requests' to add a new request.
- Each request should contain 'title' and 'priority'.
- Return responses in JSON format.
- Keep the code simple and beginner-friendly.

Code:

```
assgn15.4 task2.py > ...
1  from flask import Flask, jsonify, request
2  app = Flask(__name__)
3  requests_data = []
4  @app.route('/requests', methods=['GET'])
5  def get_requests():
6      return jsonify(requests_data)
7  @app.route('/requests', methods=['POST'])
8  def add_request():
9      data = request.get_json()
10     new_request = {
11         "title": data.get("title"),
12         "priority": data.get("priority")
13     }
14     requests_data.append(new_request)
15     return jsonify({
16         "message": "Request added successfully",
17         "data": new_request
18     })
19 # THIS PART MUST BE EXACT (no indentation)
20 if __name__ == '__main__':
21     app.run(debug=True, host='0.0.0.0', port=5000)
```

127.0.0.1:5000/requests

Pretty print

POST http

DEL http/

GET Untitl

GET http/

GET http/

>

+

▼

No environment

HTTP

http://localhost:3000/users/0

Save

POST

▼

http://10.3.6.76:5000/requests

Send

▼

Docs

Params

Auth

Headers (8)

Body

Scripts

Settings

Cookies

raw

▼

JSON

▼

Schema

Beautify

1

{

2

"title": "Internet issue",

3

"priority": "High"

4

}

Body

▼

200 OK

• 19 ms • 283 B •

{}

JSON

▼

Preview

Visualize

▼

↺

☰

🔍

📄

🔗

1

{

2

"data": {

3

"priority": "High",

4

"title": "Internet issue"

5

},

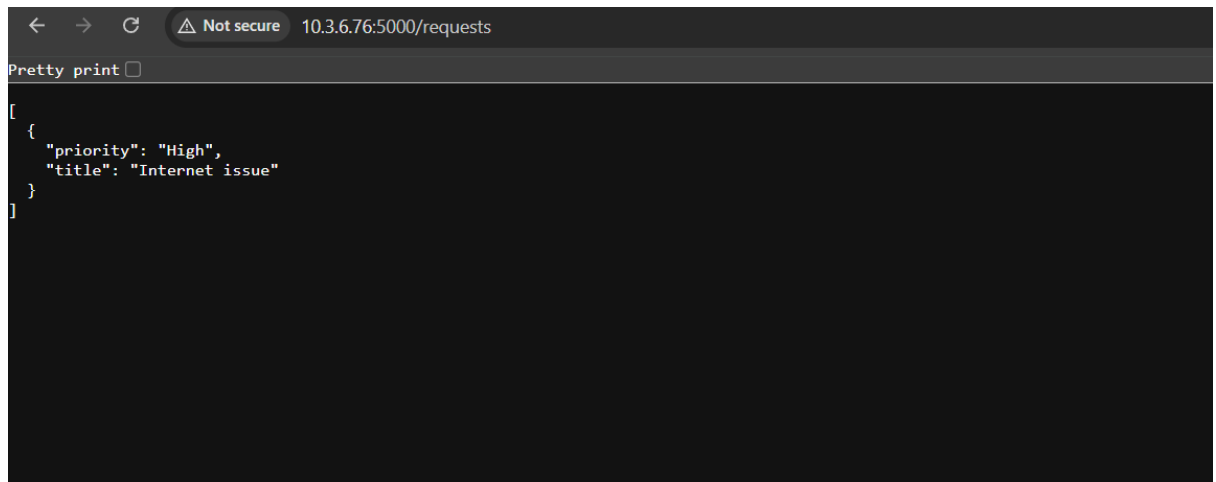
6

"message": "Request added successfully"

7

}

8

A screenshot of a web browser window. The address bar shows "10.3.6.76:5000/requests" with a "Not secure" warning. The page content displays a JSON array containing one object: [{"priority": "High", "title": "Internet issue"}]. Above the JSON, there is a "Pretty print" button with a small square icon to its right.

```
[
  {
    "priority": "High",
    "title": "Internet issue"
  }
]
```

Observation:

- The Flask API was successfully developed and executed in VS Code.
- Initially, the GET request to /requests returned an empty list.
- A POST request was used to add a new service request with title and priority.
- The API returned a confirmation message along with the added data.
- After adding data, the GET request returned the stored service request.
- This confirms that both GET and POST endpoints are functioning correctly.
- The data is stored temporarily in memory and is lost when the server restarts.

Task 3: Modify Existing Resource Using AI Assistance

Prompt:

Generate a Flask API in Python.

Requirements:

- Maintain an in-memory list of service requests.

- Each request contains 'title' and 'priority'.
- Implement a PUT endpoint '/requests/<int:index>' to update a request by index.
- If index is valid, update the request and return:

```
{"message": "Request updated", "request": updated_object}
```
- If index is invalid, return a JSON error message with HTTP status code 404.
- Keep the code simple and beginner-friendly.

Code:

```
assgn15.4 task3.py > ...
1  from flask import Flask, jsonify, request
2
3  app = Flask(__name__)
4
5  # ✅ ADD HERE 🖱️ (right after app creation)
6  @app.route('/')
7  def home():
8      return "API is running successfully"
9
10 # In-memory storage
11 requests_data = []
12
13 # GET all requests
14 @app.route('/requests', methods=['GET'])
15 def get_requests():
16     return jsonify(requests_data)
17
18 # POST add request
19 @app.route('/requests', methods=['POST'])
20 def add_request():
21     data = request.get_json()
22
23     new_request = {
24         "title": data.get("title"),
25         "priority": data.get("priority")
26     }
27
28     requests_data.append(new_request)
29
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task3.py"

* Running on http://127.0.0.1:5001

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

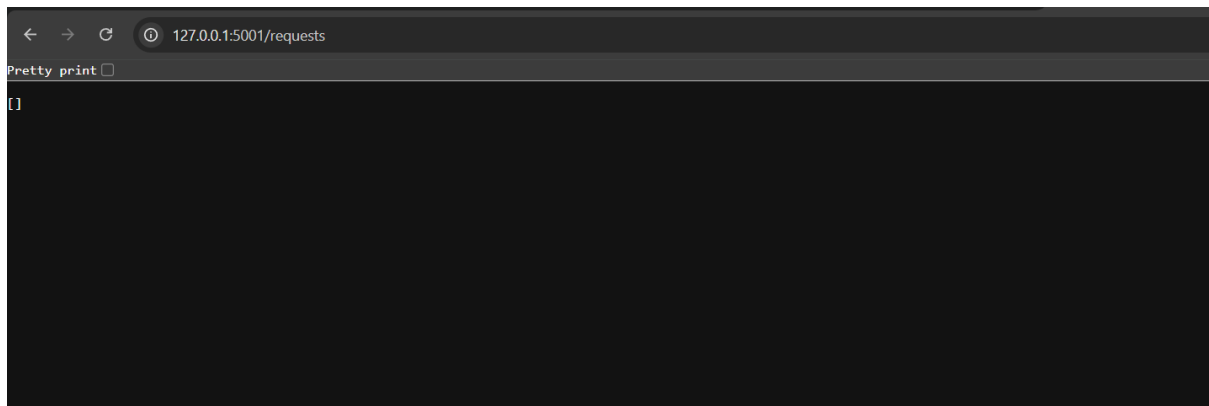
* Debugger PIN: 107-307-218

```
assgn15.4 task3.py > ...
20 def add_request():
21     """Add a new request to the list"""
22
23     new_request = {
24         "title": data.get("title"),
25         "priority": data.get("priority")
26     }
27
28     requests_data.append(new_request)
29
30     return jsonify({
31         "message": "Request added successfully",
32         "data": new_request
33     })
34
35 # PUT update request
36 @app.route('/requests/<int:index>', methods=['PUT'])
37 def update_request(index):
38     if index < 0 or index >= len(requests_data):
39         return jsonify({"error": "Request not found"}), 404
40
41     data = request.get_json()
42
43     requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
44     requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])
45
46     return jsonify({
47         "message": "Request updated",
48         "request": requests_data[index]
49     })
50
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task3.py"

* Debugger is active!
* Debugger PIN: 107-307-218
127.0.0.1 - - [24/Mar/2026 22:08:07] "GET /requests HTTP/1.1" 200 -
127.0.0.1 - - [24/Mar/2026 22:08:08] "GET /favicon.ico HTTP/1.1" 404 -



 **http://localhost:3000/users/0** Save

POST

http://127.0.0.1:5001/requests

Send

DocsParamsAuthHeaders (8)**Body**ScriptsSettings

Cookies

rawJSON

SchemaBeautify

```
1  {
2    "title": "Internet issue",
3    "priority": "High"
4  }
```

Body

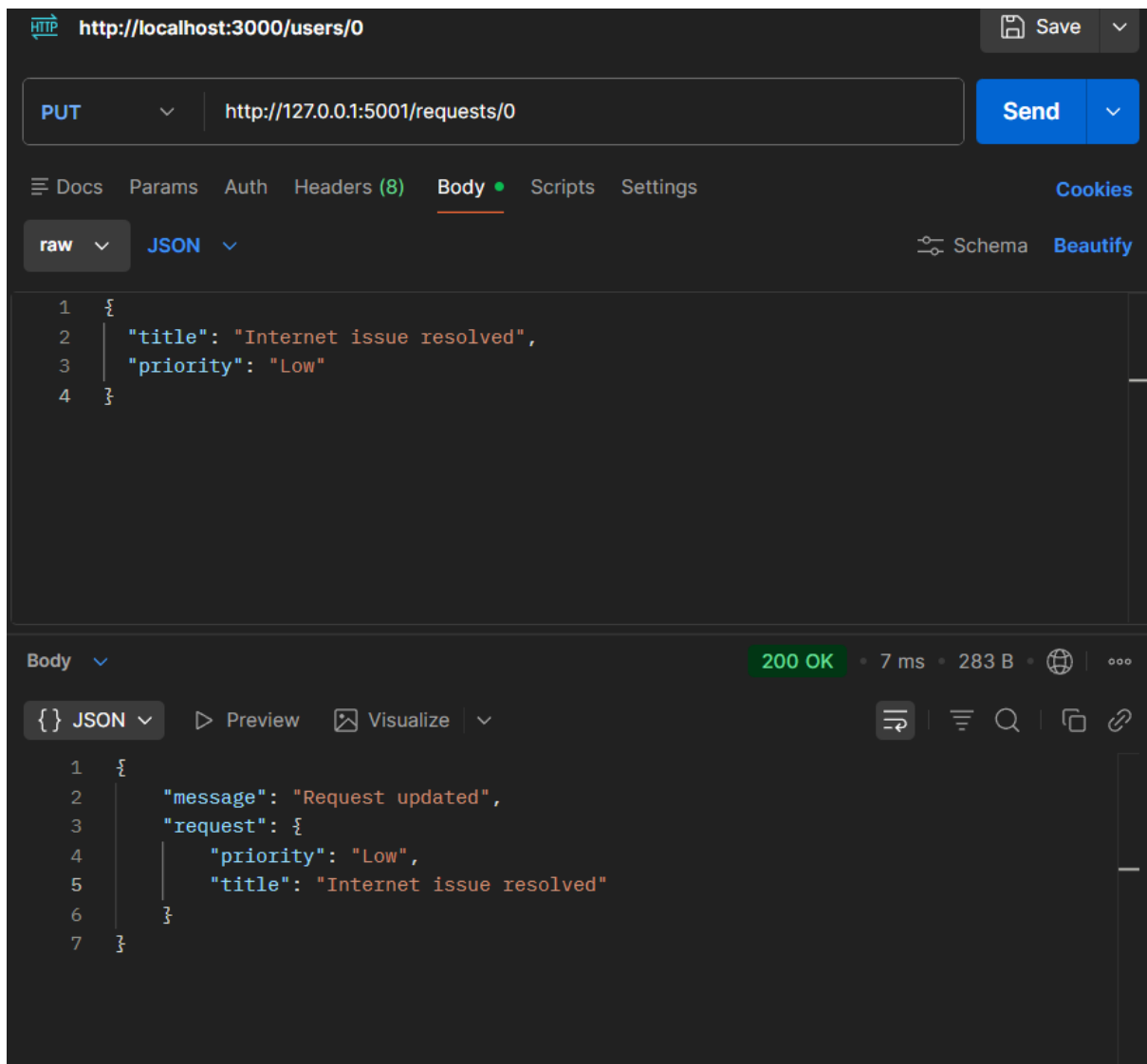
200 OK • 9 ms • 283 B •  

{ } JSON

PreviewVisualize

```
1  {
2    "data": {
3      "priority": "High",
4      "title": "Internet issue"
5    },
6    "message": "Request added successfully"
7  }
```



Observation:

- The PUT endpoint was successfully implemented to update service requests.
- When a valid index was provided, the request was updated correctly.
- The API returned a confirmation message along with the updated data.
- When an invalid index was used, the API returned a proper error message.
- The error response included HTTP status code 404, ensuring proper error handling.

- This confirms that the update functionality works correctly and handles edge cases.

Task 4: Remove a Resource Safely

Generate a Flask API in Python.

Requirements:

- Maintain an in-memory list of service requests.
- Each request contains 'title' and 'priority'.
- Implement a DELETE endpoint '/requests/<int:index>' to remove a request by index.
- If index is valid, delete the request and return:

```
{"message": "Request deleted", "request": deleted_object}
```
- If index is invalid, return a JSON error message with HTTP status code 404.
- Ensure the server does not crash on invalid input.
- Keep the code simple and beginner-friendly.

Code:

```

assgn15.4 task4.py > ...
1  from flask import Flask, jsonify, request
2
3  app = Flask(__name__)
4
5  # In-memory storage
6  requests_data = []
7
8  # Root route (optional but useful)
9  @app.route('/')
10 def home():
11     return "API is running successfully"
12
13 # GET all requests
14 @app.route('/requests', methods=['GET'])
15 def get_requests():
16     return jsonify(requests_data)
17
18 # POST add request
19 @app.route('/requests', methods=['POST'])
20 def add_request():
21     data = request.get_json()
22
23     new_request = {
24         "title": data.get("title"),
25         "priority": data.get("priority")
26     }
27
28     requests_data.append(new_request)
29

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task4.py"

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5002

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

Ln 66, Col 33 Spaces: 4 UTF-8 CRLF {} Python 3.13.7

```

assgn15.4 task4.py > ...
20 def add_request():
21     requests_data.append(new_request)
22
23     return jsonify({
24         "message": "Request added successfully",
25         "data": new_request
26     })
27
28 # PUT update request
29 @app.route('/requests/<int:index>', methods=['PUT'])
30 def update_request(index):
31     if index < 0 or index >= len(requests_data):
32         return jsonify({"error": "Request not found"}), 404
33
34     data = request.get_json()
35
36     requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
37     requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])
38
39     return jsonify({
40         "message": "Request updated",
41         "request": requests_data[index]
42     })
43
44 # DELETE request
45 @app.route('/requests/<int:index>', methods=['DELETE'])
46 def delete_request(index):
47     if index < 0 or index >= len(requests_data):
48         return jsonify({"error": "Request not found"}), 404
49
50     requests_data.pop(index)
51     return jsonify({"message": "Request deleted successfully"})
52

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assgn15.4 task4.py"

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5002

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

```

47         "message": "Request updated",
48         "request": requests_data[index]
49     })
50
51 # ☒ DELETE request
52 @app.route('/requests/<int:index>', methods=['DELETE'])
53 def delete_request(index):
54     if index < 0 or index >= len(requests_data):
55         return jsonify({"error": "Request not found"}), 404
56
57     deleted_request = requests_data.pop(index)
58
59     return jsonify({
60         "message": "Request deleted",
61         "request": deleted_request
62     })
63
64 # Run server
65 if __name__ == '__main__':
66     app.run(debug=True, port=5002)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> py "assign15.4 task4.py"

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5002

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

← → ↻ ⓘ 127.0.0.1:5002/requests

Pretty print ☐

[]

 <http://localhost:3000/users/0>

 Save

POST

http://127.0.0.1:5002/requests

Send

Docs

Params

Auth

Headers (8)

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "title": "Internet issue",
3   "priority": "High"
4 }
```

Body

200 OK • 9 ms • 283 B •  

{ } JSON

Preview

Visualize

```
1 {
2   "data": {
3     "priority": "High",
4     "title": "Internet issue"
5   },
6   "message": "Request added successfully"
7 }
```

HTTP

http://localhost:3000/users/0

Save

</>

DELETE

http://127.0.0.1:5002/requests/0

Send

↗

Docs

Params

Auth

Headers (8)

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "title": "Internet issue",
3   "priority": "High"
4 }
```

Body

200 OK

8 ms

275 B

🌐

⋮

{ } JSON

Preview

Visualize

⌵

↶

☰

🔍

📄

🔗

```
1 {
2   "message": "Request deleted",
3   "request": {
4     "priority": "High",
5     "title": "Internet issue"
6   }
7 }
```

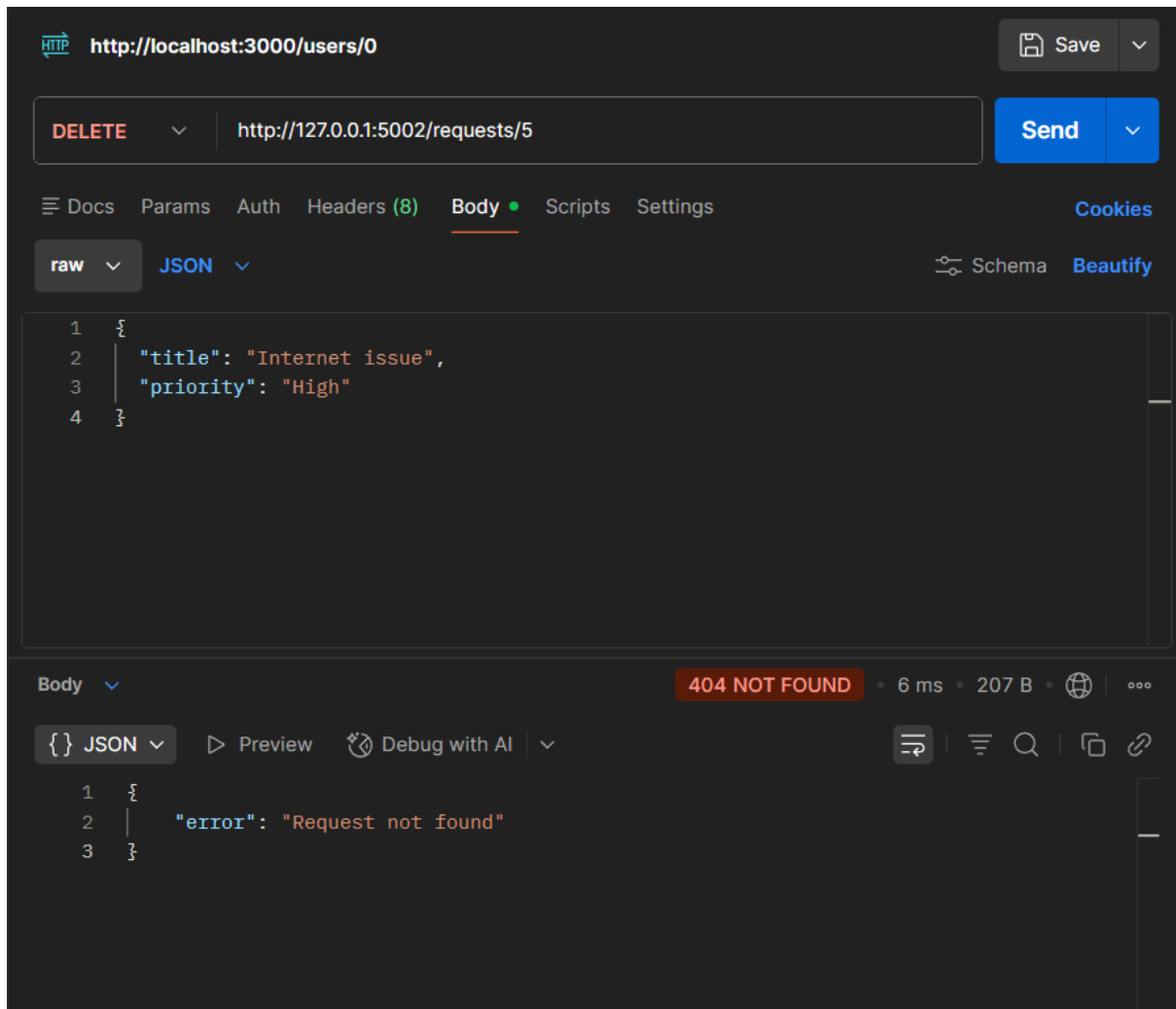
Conv

Add t

Gene

PUT

Desc



Observation:

- The DELETE endpoint was successfully implemented to remove service requests.
- When a valid index was provided, the request was deleted correctly.
- The API returned confirmation along with the deleted request details.
- When an invalid index was used, the API returned an appropriate error message.
- The server handled invalid input gracefully without crashing.
- This confirms proper functioning of the DELETE operation and error handling.

Task 5: AI-Assisted API Documentation and Readability

Prompt:

Enhance a Flask API by adding documentation.

Requirements:

- Add meaningful docstrings to all endpoints.
- Include inline comments explaining the logic.
- Clearly describe HTTP methods, inputs, and outputs.
- Improve code readability for other developers.
- Optionally suggest integration with Swagger or Flask-RESTX for API documentation.
- Keep the code simple and beginner-friendly.

Code:

```

assign13.4 task3.py 2 ...
1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 # In-memory storage for service requests
6 requests_data = []
7
8
9 @app.route('/')
10 def home():
11     """
12     Root Endpoint
13     -----
14     Method: GET
15
16     Description:
17     - Checks if the API is running.
18
19     Response:
20     - Returns a simple message indicating API status.
21     """
22     return "API is running successfully"
23
24
25 @app.route('/requests', methods=['GET'])
26 def get_requests():
27     """
28     Get All Requests
29     -----

```

```

30     Get All Requests
31     -----
32     Method: GET
33
34     Description:
35     - Retrieves all stored service requests.
36
37     Response:
38     - Returns a list of all requests in JSON format.
39     """
40     (variable) requests_data: list
41     return jsonify(requests_data)

```

```

42
43 @app.route('/requests', methods=['POST'])
44 def add_request():
45     """
46     Add New Request
47     -----
48     Method: POST
49
50     Input (JSON):
51     {
52         "title": "Issue description",
53         "priority": "High/Medium/Low"
54     }
55
56     Description:
57     - Adds a new service request to memory.

```

Description:
- Adds a new service request to memory.

Response:
- Returns confirmation and the added request.
"""

```
data = request.get_json()
```

```
new_request = {  
    "title": data.get("title"),  
    "priority": data.get("priority")  
}
```

```
requests_data.append(new_request)
```

```
return jsonify({  
    "message": "Request added successfully",  
    "data": new_request  
})
```

```
@app.route('/requests/<int:index>', methods=['PUT'])
```

```
def update_request(index):
```

```
    """
```

```
    Update Request
```

```
    -----
```

```
    Method: PUT
```

URL Parameter:

- index: Position of request in list

Input (JSON):

```
{  
    "title": "Updated title",  
    "priority": "Updated priority"  
}
```

Description:

- Updates an existing request by index.

Response:

- Success: Returns updated request
- Error: Returns 404 if index is invalid
"""

```
if index < 0 or index >= len(requests_data):  
    return jsonify({"error": "Request not found"}), 404
```

```
data = request.get_json()
```

```
# Update fields if provided
```

```
requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
```

```
requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])
```

```
return jsonify({
```

```
"""
Delete Request
-----
Method: DELETE

URL Parameter:
- index: Position of request in list

Description:
- Deletes a request from memory.

Response:
- Success: Returns deleted request
- Error: Returns 404 if index is invalid
"""

if index < 0 or index >= len(requests_data):
    return jsonify({"error": "Request not found"}), 404

deleted_request = requests_data.pop(index)

return jsonify({
    "message": "Request deleted",
    "request": deleted_request
})

# Run the Flask application
if __name__ == '__main__':
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

Observation:

- Docstrings were added to all endpoints describing functionality, inputs, and outputs.
- Inline comments improved code readability and understanding.
- Each API method (GET, POST, PUT, DELETE) is clearly documented.
- The API is now easier for other developers to use without reading full code.
- Documentation ensures better maintainability and collaboration.
- Optional tools like Swagger can further enhance API usability.